LAB / INTRODUCTION

1. OVERVIEW

1.1 Goal

The purpose of this laboratory is to familiarize you with the cross-development tools available for labs. At the same time, you will take your first steps in C programming language on a hardware target.

1.2 Hardware

The development board used at HES-SO, called STM32U5G9-DK2, will be used in the laboratories. Students can apply for a loan card for use at home.



Figure 1: The STM32U5G9-DK2 board

1.3 Software

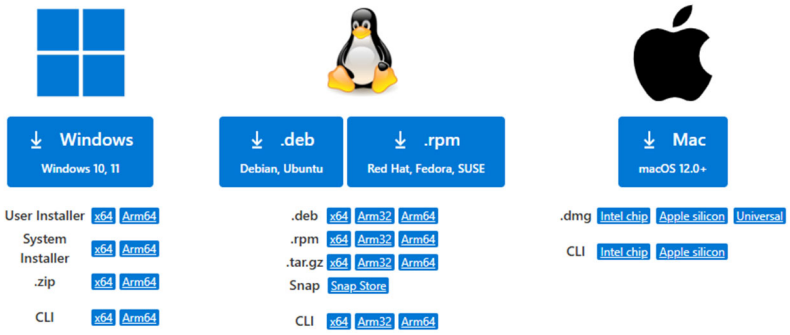
Microsoft offers a development tool called VS Code. This is the tool we will use in all the “embedded programming” laboratories.



Figure 2: The Visual Studio Code logo

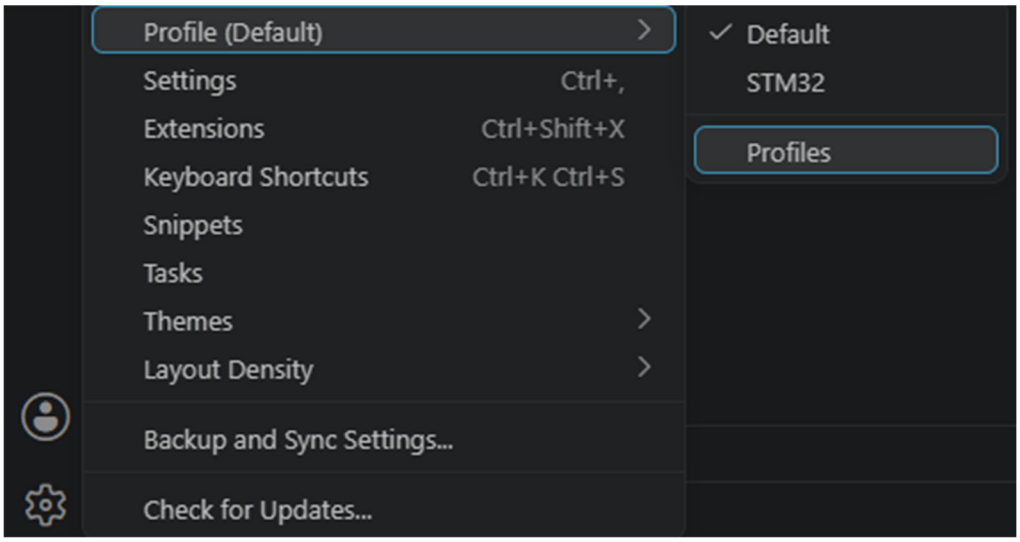
2. VS CODE INSTALLATION

If VS Code in not installed on your computer, just install it from https://code.visualstudio.com/download

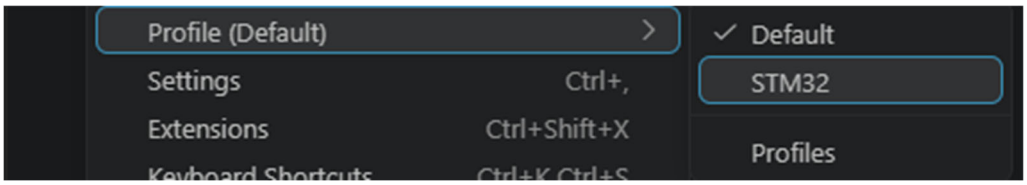


3. PROFILE CREATION

Once VSCode is installed, we will create a profile for these laboratories. Select the icon “Manage” and “Profiles”. Create a new profile (called for example STM32).



Select this profile.



4. EXTENSIONS SETUP

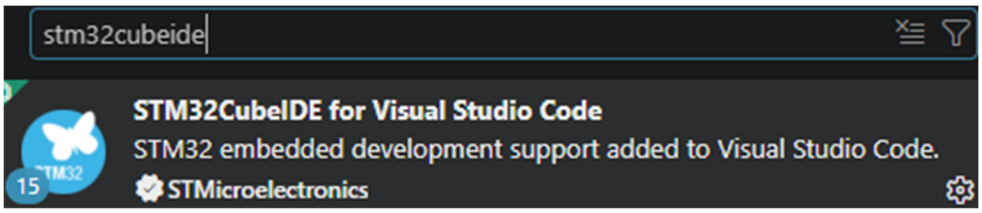
Now, you have to install the STM32CubeIDE for Visual Studio Code



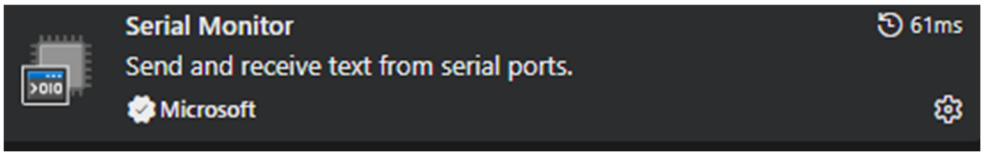
If not installed, you must select from the left panel the extension manager:



Then, type the extension name as shown below and install it.



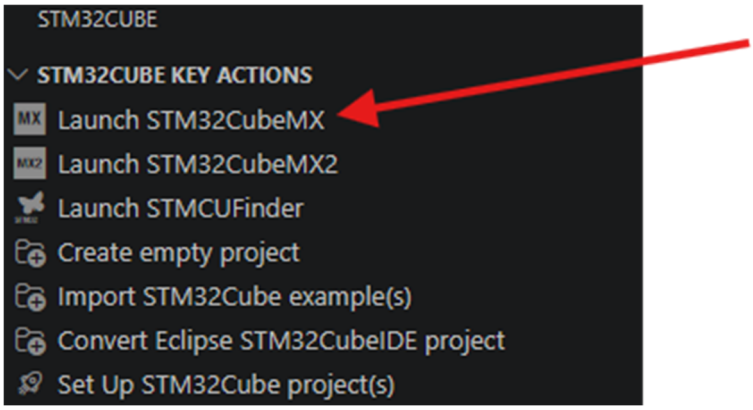
You need to install too the “Serial Monitor” to have an access to the debug serial port.



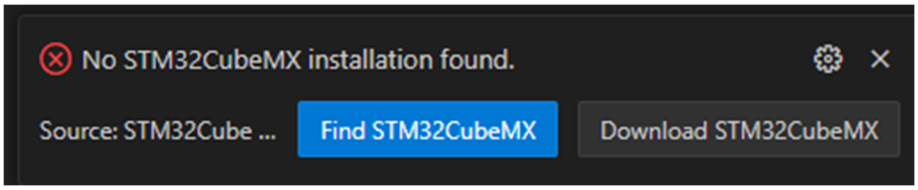
5. FIRST PROJECT CREATION

To create your first project, you must select the STM32 Cube extension on the left panel.

Then, we will start a new project from the STM32CubeMX as shown below.



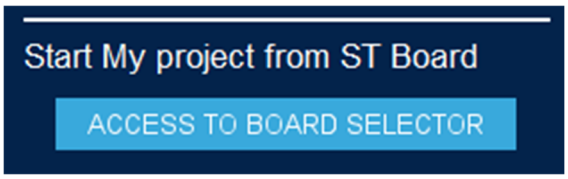
It's possible the CubeMX can't be found. On the laboratory PC, you must choose the place where STM32CubeMX is located:



C.\devel\stm32cubemx\STM32CubeMX.exe

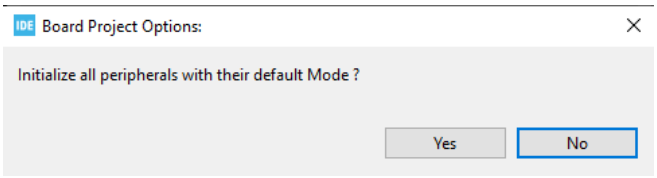
On your private PC, you must download STM32CubeMX. For that, you must create an account on ST website.

Then on STM32CubeMX, select your board.

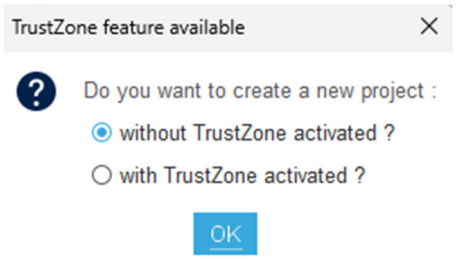




Do not initialize the peripherals with their default mode because we don't want LCD, USB, … to be configured.

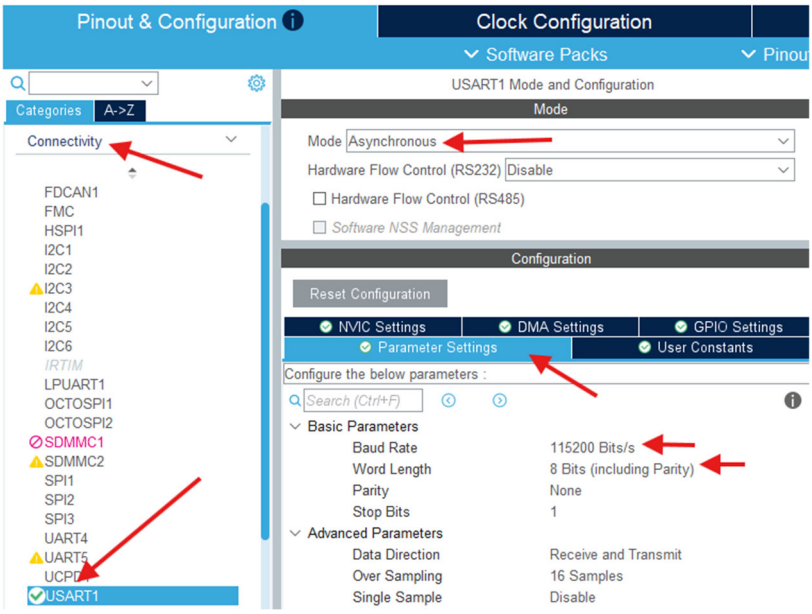


Do not choose the TrustZone activation.

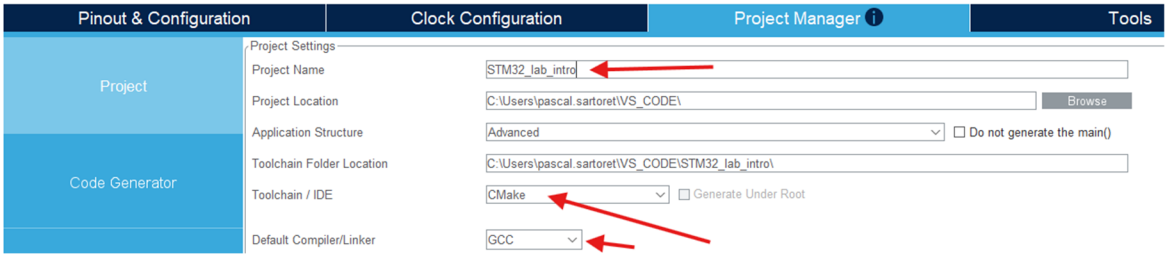


6. EXTENSION BOARD SETUP ON CUBEMX

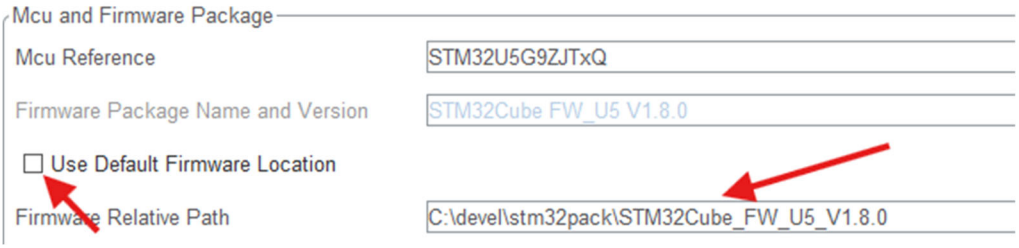
The only parameter we will change in this first laboratory is the serial port. To activate the serial debug port, select simply the parameters on the picture below.



Now, go on the project manager tab and select the project name, project location (never on a server), toolchain and compiler as shown below.



On the laboratory PC, to have not too many copies of the STM32 packs, you must choose the pack as shown below.

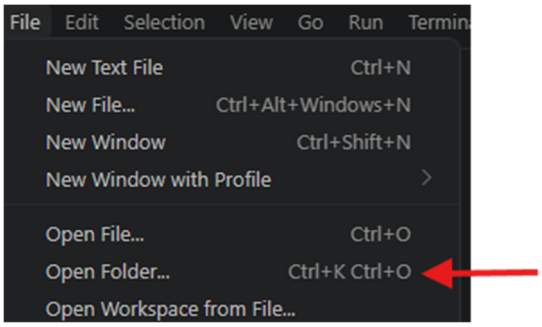


Finally, generate code with the appropriate button.

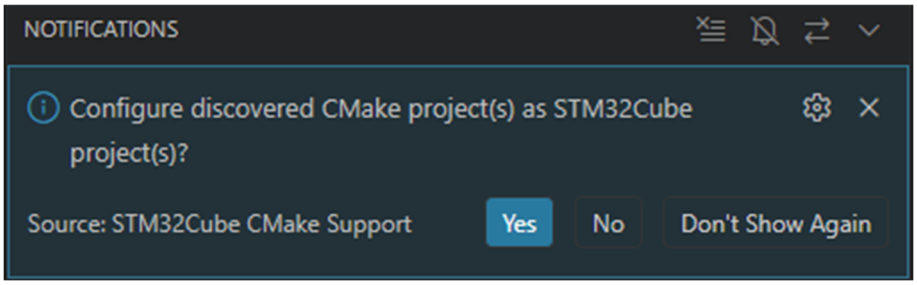


7. OPEN THE PROJECT IN VSCODE

Go back in VS Code and open the folder where you have generated your project.

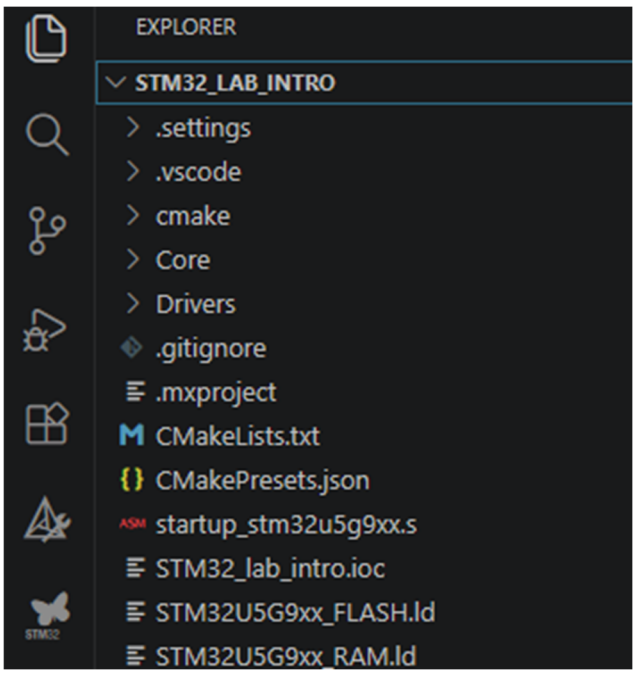


The notification below appears when you open the project.



Select “Yes”.

On VS Code explorer, you will see your project files as below.

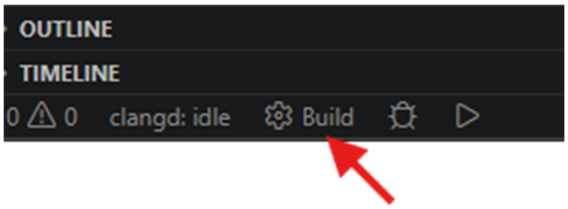


On the folder Code/Src, you will see the main.c file where we will make changes.

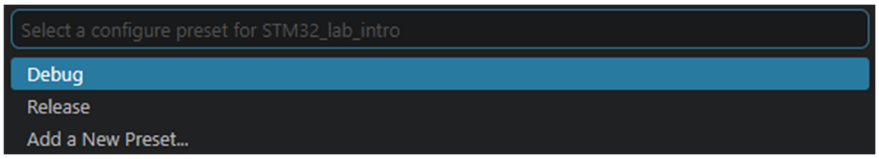


6. FIRST COMPILATION

At the bottom left of your VS Code software, you can see the “Build” button.



Click on build and at the top of can choose “Debug” for our laboratories.

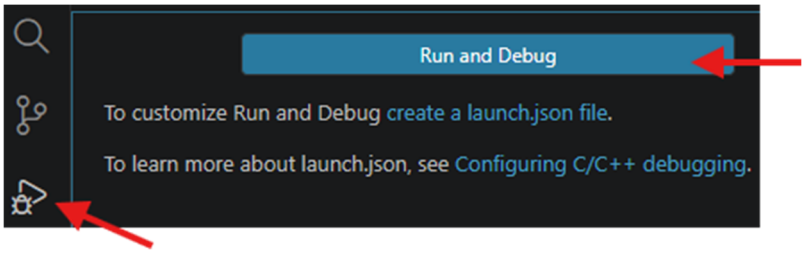


If no error exist, the build will respond to the output below.

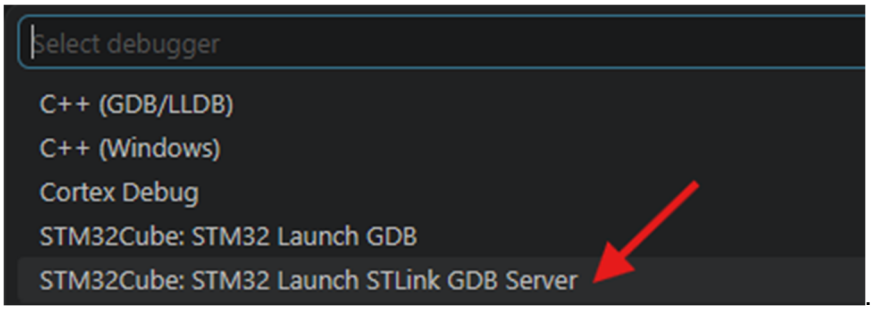
```
[build] [23/24] Building C object cmake/stm32cubemx/CMakeFiles/STM32_D
stm32u5xx_hal_dma_ex.c.obj
[build] [24/24] Linking C executable STM32_lab_intro.elf
[build] Memory region         Used Size  Region Size  %age Used
[build]              RAM:        1584 B      3008 KB      0.05%
[build]            SRAM4:           0 B        16 KB      0.00%
[build]            FLASH:       10685 B         4 MB      0.25%
[driver] Build completed: 00:00:01.265
[build] Build finished with exit code 0
```

6. FIRST DEBUG

To debug your project, select on the left panel the debug button and click on “Run and Debug”.



Now, at the top, select the debugger STLink to debug communication with your hardware



6. TASK ONE

The main.c file has been generated by the graphical tool. This file contains the initialization code of your system. Of course, you can modify it, but you must be careful to add code between the comments as an example below.

```
/* USER CODE BEGIN 2 */
```

- Add your code **here only !!!**

```
/* USER CODE END 2 */
```

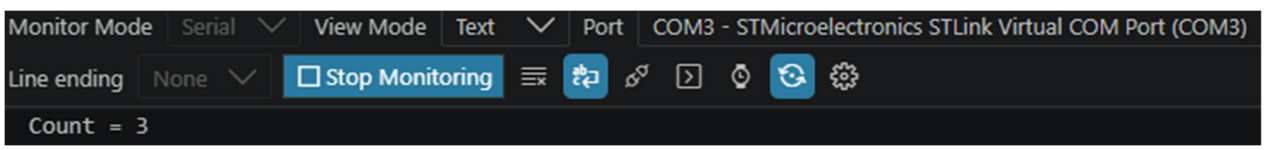
we want to display “Hello” on the “Serial Port” window we have installed previously. For that, we need to implement the serial port USART1 on the microcontroller

If we want to use `printf()` function, we need to redirect the `__io_putchar` function. Copy the code below to implement this.

```
/* USER CODE BEGIN 0 */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY); // Change
&huart1 to your active UART instance
    return ch;
}
/* USER CODE END 0 */
```

Now, you can use the `printf()` function to send something. Caution, `printf()` function places the bytes in a buffer before sending them. You must use `'\n'` to the end of the string as below to force sending of string.

```
printf("Count = %d\n",value);
```



Finally, we want use the `scanf()` function to read something from the serial port.

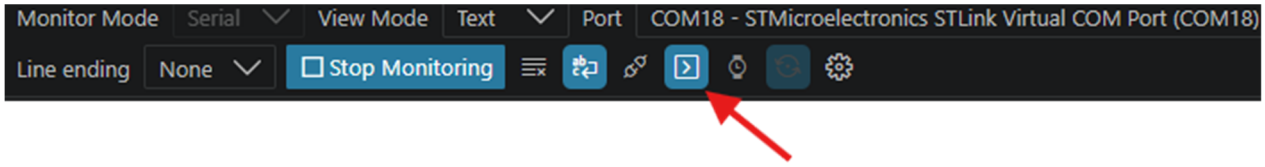
```
int __io_getchar( void )
{
    uint8_t Byte;

    HAL_UART_Receive( &huart1, &Byte,1, HAL_MAX_DELAY );
    return (int)Byte;
}
```

Caution, `scanf()` function places the bytes in a buffer before returning to the main call. The best practice is to remove the printf/scanf buffer. For that, just call this before the while(1) of the main loop.

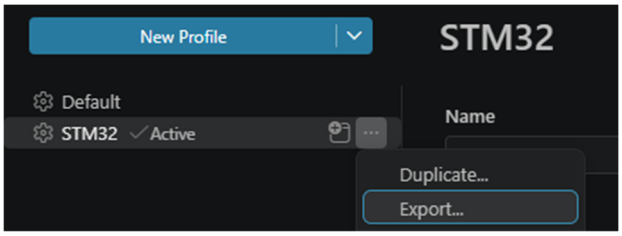
```
  setvbuf(stdin, NULL, _IONBF, 0);  // fixes scanf buffer to 0
  setvbuf(stdout, NULL, _IONBF, 0); // Optional: fixes printf  too
```

Caution, to send directly what you type on the serial port, you must activate the “terminal mode”.

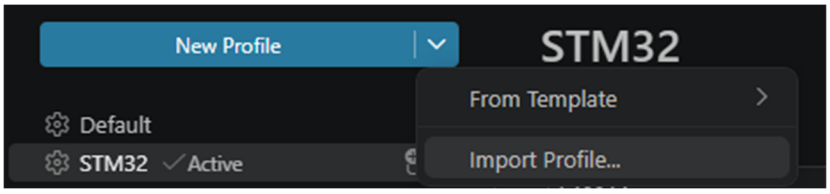


7. SAVE PROFILE

It will be nice to save your profile for another PC usage (for example). To do that, you juste need to export this profile.



When you want to use the same profile on another PC, just import it and all the extension will be automatically imported.



Keep all you made in this laboratory for further laboratories.